Ders 14 Genişletilmiş Notu: Senkron ve Asenkron Veri Bağlantı Protokolleri ve HDLC

Ders 14 Genişletilmiş Notu: Senkron ve Asenkron Veri Bağlantı Protokolleri ve HDLC

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Yeri: Hâlâ Veri Bağlantı Katmanındayız
- 2. Veri Bağlantı Denetiminin Görevleri
- 3. Asenkron Protokoller
- 4. Senkron Protokoller ve Saat Senkronizasyonu
- 5. Bayt Yönelimli Protokoller ve Denetim Karakterleri
- 6. Bayt Yönelimli Çerçeve Yapıları
- 7. Veri Saydamlığı (Data Transparency)
- 8. IBM BSC Protokolü
- 9. Bit Yönelimli Protokoller ve HDLC
- 10. HDLC Çerçeve Tipleri
- 11. Bayrak Alanı ve Bit Doldurma
- 12. Adres Alanı
- 13. Denetim Alanı ve Pencere Boyutu
- 14. I-Frame
- 15. S-Frame
- 16. U-Frame
- 17. Poll/Final Biti
- 18. HDLC İşleyiş Örnekleri
- Örnek Sorular

1. Dersin Yeri: Hâlâ Veri Bağlantı Katmanındayız

Bu ders, dönemin ikinci yarısının ilk dersidir. Vize haftasına kadar dersi Dr. Ziya Cihan Tayşi anlatmıştı. Bu dersten itibaren dönem sonuna kadar dersi Dr. Furkan Çakmak anlatıyor. Hoca, Cihan Hoca'nın akış denetimi (flow control) ve veri bağlantı denetimi tekniklerini bitirip asenkron ve senkron veri bağlantı protokollerine geldiğini söyledi. Bir hafta geride olunduğu için bu konular bu derste birleştirilerek anlatıldı. Amaç, gelecek hafta LAN teknolojilerine, yani IEEE 802 projesine başlayabilmekti.

Hoca önce kısa bir özet geçti. Dersin ilk haftalarında yedi katmanlı OSI referans modeli anlatılmıştı. Bu katmanlardan biri kablolamayla ilgili fiziksel işlemleri yapan fiziksel katmandı (physical layer). Ardından veri bağlantı katmanı (data link layer) anlatıldı. Bu hafta ve sonraki haftalarda da hep veri bağlantı katmanında kalınacak. Son haftalarda ağ katmanına (network layer) biraz girilecek, ama ağ katmanının ayrıntısı başka bir derste anlatılıyor.

Veri bağlantı katmanında veri gönderimi için çeşitli işlemler yapılır. Verinin iki nokta arasındaki akışı senkron ya da asenkron olabilir. Buna göre veri bağlantı denetim protokolleri (data link control protocols) ikiye ayrılır:

- asenkron protokoller,

- senkron protokoller:
  - bayt yönelimli (byte-oriented),
  - bit yönelimli (bit-oriented).

Bu derste ağırlık senkron protokollerdedir, özellikle bit yönelimli olanlarda. Çünkü bugün günlük hayatta kullanılan veri iletim teknikleri bunlardır. Hoca, derste kendi sesinin, görüntüsünün ve ekran paylaşımının da alt katmanlarda büyük olasılıkla senkron ve bit yönelimli bir protokolle öğrencilere ulaştığını söyledi.

Veri bağlantı denetiminin hem yazılım hem donanım tarafı vardır. Donanım tarafı daha çok elektronik ve haberleşme mühendisliğinde ayrıntılı işlenir. Bu derste yazılım, yani protokol tarafı üzerinde durulacak.

2. Veri Bağlantı Denetiminin Görevleri

Hoca veri bağlantı denetiminin görevlerini şöyle sıraladı.

Senkronizasyon. Fiziksel katmanda senkron iletim için ortak bir zamanlama işareti (common timing signal) vardı. Gerekirse bunun için ayrı bir hat kurulur ve iki uç bu işaret üzerinden senkron olur. Mantıksal seviyede ise gönderilen çerçevenin (frame) nerede başlayıp nerede bittiğini söyleyen özel bitler ya da bit dizileri kullanılır.

Akış denetimi (flow control). Noktadan noktaya (point-to-point) çalışıldığını düşünelim. Gönderici veriyi yollar, alıcı aldığını bildirirse gönderici yeniden gönderebilir. Ya da gönderici çok hızlı göndermeye, alıcı ise daha yavaş almaya uygundur. Bu durumda alıcı göndericiye yetişemez ve sorun çıkar. Bunların hepsinin denetimine akış denetimi denir.

Hata denetimi (error control). Veri karşıya ulaşmış ama hatalı ulaşmışsa, alıcının onu yeniden isteyebilmesi gibi işler.

İletim kuralları. Bazıları çok basittir: Kim gönderici, kim alıcı? Nasıl gönderilecek? Gönderilen çerçevenin tipi ne? Veri içeren bir çerçeve mi gönderiliyor, yoksa “gönderdiğin veriyi aldım” diyen bir bilgilendirme mesajı mı? Hattın işleyişiyle, iki tarafın zamanının senkronlanmasıyla ilgili çerçeve tipleri de vardır. Gönderilen çerçevenin amacının ne olduğuna dair bu kuralların hepsine iletim kuralları denir.

3. Asenkron Protokoller

Asenkron protokoller bugün günlük hayatta çok sık kullanılmaz. Önceden modemlerle yapılan bağlantılarda farklı tekniklerle ve farklı adlarla uzun yıllar kullanıldılar.

3.1 Senkron ile Asenkron Arasındaki Fark

Senkron çalışmada gönderici, çerçevenin (veri bağlantı katmanında olunduğu için “çerçeve” denir) ne zaman başlayıp ne zaman bittiğini karşı tarafa söyler. Ayrıca karşı tarafın çerçeveyi alıp almadığından haberdar olur. Haberdar olamadığı durumlar da vardır. Çerçeve yolda bir çarpışmaya (collision) uğramış ya da sinyal zayıflaması (attenuation) yüzünden karşıya doğru ulaşmamış olabilir. Hiç ulaşmamışsa alıcı, kendisine bir mesaj gönderileceğinden bile habersiz olabilir. Böyle durumlarda senkron tarafta zaman aşımı (timeout) mekanizmaları devreye girer ve çerçeve yeniden gönderilir.

Asenkron çalışmada ise hattan her an veri gelebilir. Alıcı için önemli olan verinin ne zaman geldiği değil, geldiği anda nerede başladığını bir başlangıç bitinden, nerede bittiğini de bir bitiş bitinden anlayabilmesidir. Bu, mantıksal seviyede bir denetimdir. Veri belli periyotlarla gelmez; gönderici göndermek istediğinde gelir. Bu yüzden hatta farklı uzunlukta boşluklar oluşur ve hat kullanım oranı (line utilization) düşer.

3.2 Asenkron İletimin Dezavantajları

- **Ek veri (additional data):** Gönderilecek her bayt için, ya da her iki bayt için, başına ve sonuna başlangıç ve bitiş denetimi eklenir. Hocanın örneğinde karşıya 10 bayt ulaşıyorsa bunun 3 baytı yalnız denetim için, 7 baytı veridir. Bu $3/10 = \%30$ demektir. Hoca duruma göre %30–40 arası oranlar olduğunu söyledi.

- **Yavaşlık:** Çerçeveler arasında çok fazla boşluk olabildiği için asenkron protokoller senkron protokollere göre görece yavaştır.

  Ders sonrası not: Hoca bir öğrenci sorusunu cevaplarken asenkron iletimde baytın genellikle iki başlangıç biti ve bir bitiş biti aldığını, 8 bitin 3'ünün denetime gittiğini ve kaybın %35–40 olduğunu söyledi. Hesap tutarlı değil: 8 bitin 3'ü denetimse oran $3/8 = \%37{,}5$ olur, ama o zaman veri 5 bittir. Büyük olasılıkla yaygın düzen bir başlangıç biti, 8 veri biti ve bir bitiş bitidir. Bu durumda ek yük $2/10 = \%20$ olur. Başlangıç biti her zaman tektir; bitiş biti 1, 1,5 ya da 2 bit olabilir.

4. Senkron Protokoller ve Saat Senkronizasyonu

Senkron protokollerde başlangıç ve bitiş mantıksal seviyede birer bitle değil, birer bayrakla (flag) belirtilir. Bayraklar özel dizilimlere sahip baytlardır. Bir bayt 8 bittir; bu 8 bitin belirli bir kombinasyonu, örneğin 137'ye karşılık gelen bir bayt, bir bayrağa karşılık gelir ve belirli bir anlamı vardır. Örneğin çerçevenin başına konan SYN bayrağı çerçevenin başladığını, iletim sonu (end of transmission) bayrağı bittiğini anlatır.

Senkronizasyonun iki türü vardır:

- **Fiziksel seviye:** Ortak saat işareti (common clock signal) ayrı bir hat üzerinden iki tarafın zamanını aynı tutar.
- **Mantıksal seviye:** Veri bağlantı katmanında bayraklarla yapılır. Bu dersin asıl konusu budur.

4.1 Zamanın Aynı Olması Neden Sorundur?

Hoca, iki cihazın zamanının milisaniyeden daha hassas biçimde ortak olması gerektiğini söyledi. Cihazlar illa bilgisayar değildir; robot süpürgeler, buzdolapları, kahve makineleri gibi iletişim kurduğumuz her cihazın bir zamanı vardır. Elektrik kesintisi gibi bir olay bu zamanı değiştirebilir. Kesinti hiç olmasa bile, her donanımın zamanı sayan zamanlayıcı (timer) elemanı kusursuz değildir, bir hata payıyla çalışır. Hocanın günlük hayattan örneği otomobillerdi: Otomobilin kalitesine ve kullandığı sensöre göre saati bir ayda 2–3 dakika kayabilir. İnternetten güncellenmeyen telefonlarda bile bu görülebilir.

Hoca bir sorun daha olduğunu söyledi: Saat ve dakika ölçeğinde bakınca bir gün 24 saattir, ama milisaniye ve altı hassasiyette her gün birbirinden farklı uzunluktadır ve bu farklar birikir. Bu yüzden cihazlar arasında zamanın sürekli senkron tutulması gerekir. Ortak saat işareti, iki tarafın zamanını senkronlayan bir hattır.

> **Ders sonrası not:** Hoca bu birikmeyi dört yılda bir gelen artık yılla ilişkilendirdi. Büyük olasılıkla artık yılın nedeni günün uzunluğu değil, yılın yaklaşık 365,2422 gün olmasıdır. Dünyanın dönüşündeki düzensizlikler ise artık saniye (leap second) ile düzeltilir.

4.2 Senkron İletimin Avantajı

Senkron tarafta asenkron tarafa göre daha hızlı iletim yapılır. Asenkron tarafta ortak senkronizasyonu sağlayan bir yapı olmadığı için gönderme ve alma arasındaki boşluklar fazla olabiliyordu. Senkron tarafta ise verinin ne zaman, nasıl iletildiği bilinir ve bu bir avantajdır. Bu teknolojiler yerel ağlarda (LAN), metropol ağlarda (MAN) ve geniş alan ağlarında (WAN) kullanılır. Bugün kablolu bağlandığımız Ethernet ya da kablosuz bağlandığımız Wi-Fi birer LAN teknolojisidir.

5. Bayt Yönelimli Protokoller ve Denetim Karakterleri

Bayt yönelimli bir protokolde bir bayt 8 bit olduğu için $2^8 = 256$ farklı bayrak tanımlanabilir. "İletimi başlatıyorum", "iletimi bitiriyorum", "şimdi göndereceğim bir denetim bayrağıdır", "zamanımızı ortak yapalım", "gönderdiğini almadım", "bana bir şey gönderdin mi?" gibi ifadeler için ASCII tablosundaki kodlama kullanılıyorsa 256 bayrak tanımlanabilir.

Elbette gönderilen her şey bayrak değildir. ASCII tablosundaki rakamlar, noktalama işaretleri ve harfler yazdırılabilir (printable) karakterlerdir. Enter, boşluk, null gibi ekranda göremediğimiz karakterler ise yazdırılamaz (unprintable) karakterlerdir. Bayt yönelimli tarafta ASCII tablosundaki karşılıklar doğrudan kullanıldığı için bu protokollere **karakter yönelimli (character-oriented)** de denir. Bazı kaynaklarda bu adla geçer.

5.1 Denetim Karakterleri

Hoca slayttaki denetim karakterlerini tek tek açıkladı. Protokolün yeni sürümlerinde daha fazlası olabilir. Hoca bunları bu kez ayrıntılı anlattığını, sonraki protokollerde benzer karakterler için tekrar etmeyeceğini söyledi.

- **SYN (Synchronous Idle):** Kanalın veri iletişimine müsait olmasını sağlayan denetim karakteri.
- **PAD (dolgu):** Mesaj çerçevelere bölünür. Hocanın örneği: 250 baytlık bir mesaj 40 baytlık çerçevelere bölünecek. Altı çerçeve 40'ar bayt gider ($6 \times 40 = 240$), sonda 10 baytlık bir parça kalır. Çerçeve boyu standartsa kalan 30 bayta tamamlayıcı bir dolgu uygulanır.
- **DLE (Data Link Escape):** Programlamadaki kaçış karakteri gibidir. Özel anlamı olan bir karakterin veri içinde kullanılması gerektiğinde kullanılır.
- **ENQ (Enquiry):** Sorgu. Birincil (primary) ve ikincil (secondary) cihazlar vardı. "Bana bir şey göndermek istiyor musun?" diye sorulan durumlar için kullanılır.
- **SOH (Start of Header):** Çerçevenin metin kısmına ek olarak bir başlık (header) kısmı olabilir. Başlığın nerede başladığını söyler.
- **STX (Start of Text):** Metnin, yani verinin başladığı nokta. Bu yöntemler ilk çıktığında mesaj ses ya da görüntü değil, bir A4 sayfasına yazılmış metin gibi düşünülüyordu. Bu yüzden veriye "metin" (text) denir.
- **ITB (Intermediate Text Block):** Ara bloğun sonu.
- **ETB (End of Transmission Block):** Blok sonu.
- **ETX (End of Text):** Metnin son noktası.
- **EOT (End of Transmission):** İletimi sonlandırıyorum. Hoca bunun vizede de sorulduğunu söyledi.
- **BCC (Block Check Count):** LRC kullanılıyorsa 1 bayt, CRC kullanılıyorsa 2 bayt.
- **ACK0 / ACK1:** Onay karakterleri. Dur ve bekle (stop-and-wait) yönteminde gönderici mesajı yollar, "aldım" cevabını bekler, sonra yeniden gönderir. ACK0 ile ACK1 dönüşümlü kullanılır: "0 numaralı çerçeveyi aldım, 1'i gönder", sonra "1'i aldım, sıradaki 0'ı gönder". Kayan pencere (sliding window) açısından bakınca iki pencerelik bir pencere gibi davranır.
- **Bekleme karakteri:** Hoca bunu "iletimi bekleyen", "bana bir şey göndermeni bekliyorum" demek için kullanılan bir karakter olarak anlattı.
- **Tampon karakterleri:** Tamponlanmış verileri işlemek ya da "artık tampona alıyorum" demek için kullanılan karakterler.
- **NUL (null):** Karşıya veri gönderilir ama bu veri yalnız hattı meşgul etsin, ek bir anlam taşımasın diye kullanılır.
- **Acil cevap (urgent response) karakteri:** "Senden çok acil cevap bekliyorum; bana mesaj göndermek isteyen başka makineler var, onları bekletiyorum, göndereceksen cevabını ya da paketini gönder" demek için kullanılan karakter.

Hoca, karşıya gönderilen şeylerin her zaman veri olmadığını vurguladı. Veriyi gönderebilmek için çeşitli denetimler gerekir ve bunlar denetim karakterleriyle yapılır. Bu yüzden bir **veri çerçevesi (data frame)** bir de **denetim çerçevesi (control frame)** vardır.

> **Ders sonrası not:** Hoca ETB'yi "iletimin bittiği" diye açıkladı. ETB, "End of Transmission Block", yani blok sonudur. İletimin tamamen bittiğini EOT bildirir.

6. Bayt Yönelimli Çerçeve Yapıları

6.1 Basit Veri Çerçevesi

Hoca slayttaki çerçeveyi soldan sağa anlattı. Önce hattı göndermeye müsait kılan iki SYN baytı gönderilir. Bütün çerçeveler SYN SYN ile başlar. Ardından STX gelir: Verinin başladığı nokta. "Bundan sonra gelecek baytlar veridir, ona göre değerlendir" demektir. Verinin uzunluğu pro-

tokole göre değişir; 30 bayt olduğu durumlar da, 1000 ya da 1400 bayt olduğu durumlar da vardır. Sonra ETX gelir: “Verim bitti.” Verinin karşıya doğru ulaşıp ulaşmadığını anlamak için veri bağlantı katmanında bir hata denetim tekniğiyle sona bir kod eklenir: LRC ise 1, CRC ise 2 bayt. Alıcı bu kodu kullanarak verinin doğru ulaşıp ulaşmadığını denetler.

```
SYN SYN STX [veri] ETX BCC
```

6.2 Başlıklı Veri Çerçevesi

Başlık kullanılan durumda başlangıç karakterlerinden sonra SOH gelir. Başlık, veriye dair içsel bilgiler taşıyabilir. Ardından metin başlar, veri gönderilir, veri biter ve hata denetim baytları gelir.

```
SYN SYN SOH [başlık] STX [veri] ETX BCC
```

6.3 Ara Bloklu Çerçeve

Bazı çerçeveler protokole göre çok uzundur. Çok uzun çerçevelerde hata denetim tekniklerini uygulamak işlemsel olarak zaman alabilir, hele donanım kısıtlıysa. Hoca, 16. ya da 32. dereceden bir polinomla çalışmanın tamamen hesaplama gücünden yediğini söyledi. Örneğin 1000 baytlık veri için 32’lik yerine 16’lık bir CRC kullanılmak isteniyorsa 1000 bayt fazla gelir. Bu yüzden veri daha küçük parçalara, ara bloklara (intermediate block) bölünür ve her bloğun ayrı denetim baytları olur. Hocanın örneği: 100 baytlık veriyi 25’er baytlık 4 bloğa bölmek.

Slayttaki örnekte iki SYN gönderilir, sonra başlığın başladığı (SOH), sonra verinin başladığı (STX) söylenir. İlk blok gönderilir ve ITB ile biter, ardından o bloğun hata denetim kısmı gelir. Sonra yeniden STX ile ikinci blok başlar ve ETB ile biter, ardından yine denetim kısmı gelir. Bu örnekte veri iki bloğa bölünmüştür; daha fazla bloğa da bölünebilir.

```
SYN SYN SOH [başlık] STX [blok 1] ITB BCC STX [blok 2] ETB BCC
```

Alıcı bu çerçeveyi alınca karşı tarafa ACK1 gönderir: “0 numaralı çerçeveyi aldım, 1’i bekliyorum.” Bu cevap SYN SYN ACK1 olmak üzere 3 baytlık bir çerçevedir. Gönderici sonra verinin devamını gönderir. Bu ikinci çerçeve de başlıkla başlar, ara blok ve hata denetim kısmı yine vardır. Tek farkı sonunun ETX ile bitmesidir. Alıcı bu kez “1’i aldım, 0’ı gönder” der (ACK0).

Hoca slaytta ikinci çerçevenin başında bir yazım hatası olduğunu, orasının başlık olması gerektiğini söyledi.

Hoca sınıfa sordu: ETB ile biten bir çerçeveyi aldığımızda ne anlarız? Cevap: Gönderici veri gönderiyor ama daha da gönderecek, çünkü çerçevenin sonu ETX değil. ETX ile biten çerçeve gelince metin bitmiştir.

Soru: Her blok için başlık bilgileri farklı mı? **Cevap:** Başlığın içindekiler verinin içeriğiyle ilgilidir. Veriye göre aynı da olabilir, farklı da. Bununla ilgili net bir şey söylenemez.

6.4 Denetim Çerçevesi

Denetim çerçevesi de başlangıç için iki SYN karakteriyle başlar. Ardından hangi denetim bilgisi gönderilecekse o gelir; uzunluğu değişebilir. Bu da sonuçta bir veri olduğu için sonunda bir hata denetim bayt grubu bulunur.

```
SYN SYN [denetim karakterleri] BCC
```

7. Veri Saydamlığı (Data Transparency)

> **Hocanın vurgusu:** Veri saydamlığı çok önemli bir kavramdır ve farklı zamanlarda karşınıza çıkacaktır. Lütfen iyi dinleyin.

Bu teknikler geliştirildiğinde genellikle metin gönderiliyordu. Kullanılan şeyler rakamlar, harfler ve noktalama işaretleriydi. ASCII tablosundaki geri kalan yazdırılamaz karakterlerin veri içinde kullanılmasına gerek yoktu. Denetim karakterleri de genelde bu yazdırılamaz, metinde geçmeyecek karakterlerden seçilmişti.

Zamanla ses ve görüntü verisi gönderilmeye başlandı. Örneğin gri tonlu bir resimde 0'dan 255'e kadar her değer grinin farklı bir tonunu ifade eder. Öyle bir tona denk gelinebilir ki verinin içindeki bir bayt bir denetim karakterine karşılık gelir. Slayttaki örnekte veri içindeki bir bayt ETX karakterine denk gelmişti. Oysa bu bayt verinin bir parçasıdır, denetim karakteri olarak yorumlanmamalıdır. Bu sorunla karşılaşma durumuna veri saydamlığı sorunu denir.

7.1 DLE ile Çözüm

Çözüm kaçış karakteri kullanmaktır. Denetim anlamı taşıyan STX, ETX, ETB, ITB ve SOH karakterlerinin önüne DLE konur. Böylece alıcı şöyle karar verir:

- Önünde DLE olmayan bir ETX gelirse bu bir veri baytıdır, denetim karakteri değildir.
- Önünde DLE varsa, ardından gelen bir denetim karakteridir.

```
DLE STX [veri] DLE ETX
```

7.2 Veride DLE Geçerse

Hoca bunun "belki milyonda bir ihtimal" olduğunu söyledi, ama verinin içinde bir kaçış karakteri ve ardından bir denetim karakteri, yani DLE ETX denk gelebilir. Bu durumda gönderici bunun başına bir DLE daha koyar. Alıcı ilk DLE'yi görünce ardından gelenlerin verinin elemanları olduğunu anlar. İlk DLE verinin elemanı değildir, gönderici onu oraya kendisi yerleştirmiştir. Gönderici, veriyi göndermeden önce baytları tek tek inceleyerek böyle durumlar olup olmadığına bakar. Bunu yapabilen göndericidir, çünkü verinin tam olarak ne olduğunu gönderen bilir, alan bilmeyebilir.

> **Ders sonrası not:** Hoca, verinin içinde üç DLE art arda gelirse "bir tane daha", dört tane gelirse yine "bir tane daha" eklendiğini söyledi. Tek DLE eklemek yetmez. Python ile denendi: Veri `A DLE DLE ETX B` iken tek DLE eklenirse çerçeve `DLE STX A DLE DLE DLE ETX B DLE ETX` olur. Alıcı ilk iki DLE'yi bir veri DLE'si olarak alır, ama sonraki `DLE ETX`'i çerçeve sonu sanar ve veriyi `A DLE` diye keser. Büyük olasılıkla BSC'deki kural, verideki her DLE'nin ikilenmesidir (DLE → DLE DLE). Bu durumda aynı veri `DLE STX A DLE DLE DLE DLE ETX B DLE ETX` olarak gider ve alıcı onu eksiksiz geri çıkarır.

8. IBM BSC Protokolü

Bayt yönelimli tekniklerden IBM'in geliştirdiği **BSC (Binary Synchronous Communication)** 1960'ların sonlarına doğru geliştirildi. Bayt yönelimli senkron protokollerin en popüleri olduğu için hoca onun özelliklerini anlattı.

- **Noktadan noktaya ve çok noktalı (multipoint):** Noktadan noktaya hatta yalnız iki taraf vardır. Biri gönderir, öbürü alır. Bu tekniklerde alıcının ya da göndericinin adresine gerek yoktur, çünkü zaten iki taraf vardır. Çok noktalı çalışmada ise karşıda örneğin 15 taraf vardır. Birine gönderilecekse onun adresi söylenmeli, ayrıca gönderen kendi adresini de yazmalıdır ki alıcı mesajın kimden geldiğini bilsin.

  Hocanın vurgusu: Bu basit görünüyor ama vizede bu soruda çok hata yapıldı. Noktadan noktaya hatta adrese gerek yoktur; çok noktalı hatta hem alıcının hem göndericinin adresi gerekir.

- **Yarı çift yönlü (half-duplex):** Bir taraf veri gönderirken öbür taraf aynı anda gönderemez. Hocanın benzetmesi polis telsiziydi: Telsizin tuşuna basan konuşur, o sırada öbürü konuşamaz; biri bitirince öbürü basıp konuşur. Tam çift yönlü (duplex) ise telefon konuşmasıdır: Bir taraf konuşurken öbürü de konuşabilir.
- **Dur ve bekle akış denetimi ve dur ve bekle ARQ (Automatic Repeat Request)** kullanılır.
- **Kod bağımlıdır:** Denetim bilgileri birer karakterdir ve protokol karakterlere bağlıdır. ASCII kullanılıyorsa 256 karakter vardır; söylenmek istenen her şey yalnız bunlarla söylenmelidir.

9. Bit Yönelimli Protokoller ve HDLC

Bit yönelimli tarafta daha ayrıntılı duruldu. Bit yönelimli protokollerde veri gönderim hızı ve çerçeveleme işlemleri daha etkin yapıldığı için bunlar daha yaygındır. Karakter tabanlı olmadıkları için veri saydamlığı sorunuyla neredeyse hiç karşılaşılmaz. Karşılaşılan bazı özel durumlar vardır; bunlar bayrak alanında anlatıldı.

9.1 HDLC'nin Özellikleri

En yaygın bit yönelimli protokol **HDLC (High-Level Data Link Control)**'dür. HDLC bir ISO standardıdır. Sonradan çıkan birçok yöntem onu temel alır ve üzerine çeşitli özellikler ekler. Hoca, derste de büyük olasılıkla bunlardan biriyle iletişim kurulduğunu söyledi.

- Noktadan noktaya ve çok noktalı çalışabilir.
- Tam çift yönlü ve yarı çift yönlü çalışabilir.
- Kayan pencere akış denetimi ve kayan pencere ARQ ile kullanılır.
- Kod bağımsızdır: Bit seviyesinde, 0 ve 1 seviyesinde çalışıldığı için ASCII kodlarına bağlı değildir.

Hoca burada BSC ile farkı bir kez daha açıkladı. BSC dur ve bekle kullandığı için iki onay karakteri (ACK0, ACK1) yeterlidir. Kayan pencerede ACK0 ve ACK1 yetmez; pencere boyu 15 ise "ACK2 nerede?" diye sorulur. Dur ve bekle yönteminde her seferinde tek çerçeve gönderilir ve hat kullanım oranının neden düşük olduğu buradan anlaşılır. Hocanın örneği: 1 kilometrelik bir hatta aynı anda yalnız bir çerçeve vardır. Çerçeve gider, alıcı alır, işler, "aldım" cevabını yollar. Kayan pencerede ise pencere 20 ise 19 çerçeve hiç onay beklemeden hatta gönderilebilir.

> **Ders sonrası not:** Hoca HDLC'yi anlatırken bir yerde "byte-oriented çalıştığımız durumda HDLC" dedi. HDLC bit yönelimlidir. Hocanın "synchronization data link control" diye andığı protokol, büyük olasılıkla IBM'in SDLC (Synchronous Data Link Control) protokolüdür; HDLC bu protokolden türetilmiştir.

Hoca, HDLC üzerine eklenerek oluşturulmuş birçok bağlantı erişim prosedürü (link access procedure, LAP) olduğunu, ama bu sunumda daha çok HDLC üzerinde durulacağını söyledi. HDLC, OSI gibi kavramsal bir yapıdır: Hayata geçirilecek tekniklerin ondan esinlenmesi için geliştirilmiştir ve çoğu onu temel alır.

> **Ek bilgi (derste anlatılmadı):** HDLC tabanlı yaygın protokollere büyük olasılıkla şu örnekler verilebilir: X.25'in veri bağlantı katmanı olan LAPB, ISDN'in D kanalında kullanılan LAPD ve noktadan noktaya bağlantılarda kullanılan PPP'nin çerçeve yapısı. Bunların hepsi HDLC'nin bayrak, adres, denetim ve FCS alanlarını az çok korur.

9.2 İstasyon Tipleri

Her noktaya bir istasyon dersek, istasyonlar şunlar olabilir:

- **Birincil (primary):** Ağ içinde olayın akışını yönetir.
- **İkincil (secondary):** Hocanın benzetmesiyle, akışın içinde "aktör olmasa da figüran" olan cihazlar.
- **Birleşik (combined):** Duruma göre birincil, duruma göre ikincil olabilir.

9.3 Yapılandırmalar

- **Dengesiz (unbalanced):** Birincil ve ikincil yaklaşımı. Birincilde yük ve yetki fazlalığı vardır, ikincilde daha azdır.
- **Dengeli (balanced):** Herkesin birleşik olduğu durumda yük eşit dağılabilir.

  > **Ders sonrası not:** Hoca dengeli yapılandırmanın HDLC'nin varsayılan sürümünde tanımlı olmadığını, yalnız dengesiz yapılandırmanın tanımlı olduğunu söyledi. Büyük olasılıkla bu doğru değildir: Dengeli yapılandırma ve ona ait ABM, HDLC'nin standart parçasıdır. LAPB gibi yaygın protokoller doğrudan ABM kullanır.

9.4 Üç Çalışma Modu

- **NRM (Normal Response Mode):** Dengesiz bir moddur. Birincil ile ikinciller arasındaki iş poll ve select kavramlarıyla yürür. Birincil, ikincillerden birinin kendisine veri göndermek isteyip istemediğini sorar; bu **poll** etmektir. Örneğin 5 ikincil varsa 3 numaraya gidip "bana bir şey göndermek istiyor musun?" der. Ya da 5 numarayı seçip "sana bir şey göndereceğim" der; bu **select** etmektir.
- **ARM (Asynchronous Response Mode):** Hat müsaitse ikinciller birincilden izin almadan iletime başlayabilir. Hat müsait değilse "sen konuş, sen sus" diye birincil karar verir.
- **ABM (Asynchronous Balanced Mode):** Bütün cihazların tipi birleşiktir ve hepsi hem birincil hem ikincil gibi davranma yetkisine sahiptir.

  Ders sonrası not: Hoca ARM'de ikincillerin birincilden onay almadan kendi aralarında konuşabildiğini söyledi. Büyük olasılıkla ARM'de ikincil izin beklemeden iletim başlatabilir, ama yine yalnız birincille haberleşir; ikinciller doğrudan birbirleriyle konuşmaz. Hoca dersin sonunda bir öğrenci sorusuna verdiği cevapta da A'nın B'ye veriyi birincil üzerinden gönderdiğini söyledi.

10. HDLC Çerçeve Tipleri

Hoca ayrıntıya geçmeden önce üç çerçeve tipini özetledi.

- **I-frame (information):** Gönderilecek veri I-frame'in içine konup karşıya gönderilir. Başına ve sonuna çeşitli bitler eklenir.
- **S-frame (supervisory):** Hata denetimi ve akış denetimiyle ilgili denetim işlevleri içindir. Örneğin "gönderdiğini aldım" S-frame ile söylenir. Karşı taraf I-frame ile veri gönderir, alıcı "aldım" mesajını S-frame ile yollar.
- **U-frame (unnumbered):** Hattın yönetimi ve sürdürülmesiyle ilgili özel tanımlı yönetim çerçeveleri. HDLC'de 32 çeşit U-frame vardır.

Piggybacking (sırtlama). Cihan Hoca'nın anlatımından hatırlatma: Alıcı, "aldım" bilgisini kendi gönderdiği mesajın içine saklayarak gönderiyorsa buna piggybacking denir. Bu, yerden, yani hattan tasarruftur. Bu durumda onay da I-frame ile gelir. Alıcının gönderecek verisi yoksa akış "gönderici I-frame, alıcı S-frame, gönderici I-frame, alıcı S-frame" diye gider.

10.1 Genel Çerçeve Yapısı

HDLC noktadan noktaya ve çok noktalı çalışmaya elverişli olduğu için çerçevede adres bilgisi de vardır.

```
Bayrak | Adres | Denetim | Bilgi | FCS | Bayrak
```

- **Bayrak:** Veri göndermeye başlandığını söyler. Bayt olmak zorunda değildir; bir bit dizisidir.
- **Adres:** Nereye gittiği.
- **Denetim:** Denetimle ilgili işlevler.
- **Bilgi (information):** Veri. I-frame'de çok uzun olabilir, yüzlerce bit olabilir.
- **FCS ve kapanış bayrağı:** Hata denetimi ve çerçevenin sonlandırılmasıyla ilgili bit dizisi.

S-frame'de bilgi alanı yoktur; yalnız karşı tarafı bilgilendirme amaçlı alanlar vardır. U-frame'de yapılacak yönetim işinin ne olduğu denetim alanıyla birlikte belirtilir. Hoca U-frame'de denetim alanının bir bayt olduğunu, yönetim bilgisinin denetimle birleştiğini söyledi.

11. Bayrak Alanı ve Bit Doldurma

Bayrak sabit bir dizidir: bir 0, altı tane 1, sonra yine bir 0.

```
01111110
```

Bayrak, çerçevenin başlangıç ve bitiş noktasını belli etmek, dolayısıyla senkronizasyon için kullanılır.

Karakter yönelimli protokollerde bütün denetim karakterlerine bayrak deniyordu ve veri içinde geçen karakterler yüzünden veri saydamlığı sorunu vardı. Burada da benzer bir sorun olabilir: Verinin içinde bayrakla aynı bit dizisi oluşabilir. Bu sorun bit doldurma (bit stuffing) ile çözülür.

Hocanın anlattığı alıcı kuralları şunlardır:

- Bir 0'dan sonra beş tane 1 gelmiş ve yedinci bit 0 ise bu 0 alınır, yani atılır. Gelenler verinin parçasıdır; zaten altı tane 1 gelmemiştir.
- Beş tane 1'den sonra bir 1 daha gelir ve ardından 0 gelirse bu bir bayraktır ve çerçevenin sonlandığı anlamına gelir.

Gönderici tarafındaki karşılığı şudur: Gönderici veride art arda beş tane 1 gördüğünde araya bir 0 ekler. Böylece veride hiçbir zaman altı tane 1 art arda görünmez. Bu kural python ile rastgele bit dizileri üzerinde denendi: Doldurulmuş dizide hiç 111111 kalmadı ve alıcı kuralı veriyi her seferinde eksiksiz geri çıkardı.

> **Ders sonrası not:** Hoca üçüncü durumu şöyle anlattı: Bir 0, beş tane 1 ve ardından iki 1 daha gelirse, yeni bir 1 görülene kadar sıfırlar sayılır; sayı 15'ten küçükse veridir, büyükse hat müsaittir. Bu anlatım doğru değildir. Sayılan sıfırlar değil, art arda gelen 1'lerdir. Büyük olasılıkla HDLC'de art arda 7 ile 14 arası 1, çerçevenin iptal edildiği (abort) anlamına gelir; 15 ya da daha fazla art arda 1 ise hattın boşta (idle) olduğunu gösterir. Bit doldurma yüzünden veride altı 1 art arda hiç geçemeyeceği için bu diziler veriyle karışmaz.

12. Adres Alanı

Adres alanı ikincil cihazların adreslerini içerir.

Tek baytlık adreslemede sekiz bitin sonuncusu her zaman 1'dir. Adresleme 7 bitle yapılır. Böylece ağda $2^7 = 128$ cihaza adres verilebilir.

İki baytlık adres de kullanılabilir. İlk 7 bit, ardından bir 0, sonra 7 bit daha. Toplam 14 bitlik adres alanı ile ağda $2^{14} = 16384$ cihazla iletişim kurulabilir (ara adım eklendi). Baytın son biti 0 ise adres bir sonraki baytta devam eder, 1 ise adres o baytta biter.

> **Ders sonrası not:** Hoca iki baytlık adresi anlatırken "7 byte" ve "14 byte" dedi. Doğrusu 7 bit ve 14 bittir.

> **Ders sonrası not:** Hoca HDLC adres alanındaki adresin ağ katmanındaki adres olduğunu, fiziksel katmandaki MAC adresinin kastedilmediğini söyledi. Büyük olasılıkla bu doğru değildir. HDLC adresi, veri bağlantı katmanında aynı hattaki ikincil istasyonu belirten yerel bir adrestir; ağ katmanı adresi değildir. MAC adresi de fiziksel katmanın değil, veri bağlantı katmanının (MAC alt katmanının) adresidir.

13. Denetim Alanı ve Pencere Boyutu

Denetim alanında gönderilen çerçevenin bir komut mu yoksa bir cevap mı olduğu ve çerçevenin tipi bildirilir.

Denetim alanı duruma göre 1 ya da 2 bayttır. Kayan pencerenin boyu en çok 7 ise denetim alanı 1 bayt, 127 ise 2 bayttır. WAN gibi gecikmesi yüksek (high delay) uygulamalarda 127 çerçevelik pencere mantıklı olur.

Hoca pencere boyunun nelere bağlı olduğunu açıkladı: kullanılan gönderim tekniği, hattın fiziksel kapasitesi, hatayla karşılaşma oranı ve mesafe. Pencere büyükse ve hatta hata alma olasılığı çok yüksekse pencereyi zaman zaman küçültmek daha uygun olabilir. Bununla ilgili önceden belirlenmiş tanımlar olabilir ve yöntemden yönteme değişir.

> **Ek bilgi (derste anlatılmadı):** Sıra numarası k bitse 2^k farklı numara vardır. Go-Back-N'de pencere en çok $2^k - 1$ olabilir; aksi hâlde alıcı, kaybolan bir pencerenin yeniden gönderilmesini yeni bir pencereyle karıştırabilir. Bu yüzden 3 bitlik numarada ($2^3 = 8$ numara) pencere 7, 7 bitlik numarada ($2^7 = 128$ numara) pencere 127'dir.

13.1 Çerçeve Tipinin Belirlenmesi

Denetim alanının başı çerçevenin tipini söyler:

- İlk bit 0 ise **I-frame**.
- İlk iki bit `10` ise **S-frame**.
- İlk iki bit `11` ise **U-frame**.

Üç çerçeve tipi olduğu için bu üç durum en çok 2 bitle ayırt edilir.

> **Ders sonrası not:** Hoca bir an "ilk biti 1, ikinci biti 0 ise bu bir I-frame" dedi, hemen ardından doğrusunu söyledi. Doğrusu: İlk bit 0 ise I-frame, `10` ise S-frame, `11` ise U-frame.

14. I-Frame

Pencere boyu 7 iken denetim alanı bir bayttır. İlk bit 0'dır ve bunun bir I-frame olduğunu gösterir. Geri kalanlar şöyledir: iki tane 3 bitlik akış denetim alanı ve bir tane P/F biti. Toplam $1 + 1 = 2$ bit çıkınca geriye 6 bit kalır; 3'ü bir alana, 3'ü öbür alana gider.

```
0 | N(S) 3 bit | P/F | N(R) 3 bit
```

- **N(S):** Gönderilen çerçevenin sıra numarası.
- **N(R):** Alıcı tarafın numarası. Son çerçeve hatasız geldiyse bir sonraki beklenen çerçevenin numarası yazılır. Son alınan çerçevede hata varsa, hata hangi çerçevedeyse onun numarası yazılır. Örneğin buraya 7 yazılmışsa "6'ya kadar olan çerçeveleri aldım, bana 7'yi yolla" denmektedir.

Yani veri gönderilen çerçevenin denetim bloğu 0 ile başladığı için onun I-frame olduğu anlaşılır ve geri kalan kısımlar piggybacking için de kullanılır.

3 bitlik alanla 0, 1, 2, 3, 4, 5, 6, 7 ifade edilebilir. Pencere 127 ise denetim alanı 2 bayttır ve numaralar iki tane 7 bitlik alanda tutulur. Daha fazla çerçeve numaralanması gerektiği için daha fazla alan vardır; işlem aynı şekilde yürür.

> **Ders sonrası not:** Hoca bir yerde ACK bilgisinin N(S) alanına yazıldığını söyledi. Onay (piggyback ACK) N(R) alanında taşınır; N(S) gönderilen çerçevenin kendi numarasıdır. Hoca ayrıca P/F bitini önce "parity" diye andı, sonra "poll ile ilgili bir bit" diye düzeltti.

15. S-Frame

S-frame'in temel amacı "gönderdiğin mesajı aldım", "almadım" ya da "şu mesajı hâlâ bekliyorum, neden göndermiyorsun?" gibi işlerdir. İlk iki biti standart olarak `10`'dır. Ardından 2 bitlik bir kod, bir P/F biti ve N(R) alanı gelir.

```
1 0 | kod 2 bit | P/F | N(R) 3 bit
```

Hoca, gönderilen metnin bitip bitmediğinin S-frame'deki ya da I-frame'deki P/F bitinden de anlaşılabildiğini söyledi.

15.1 S-Frame Kodları

Kod	Ad	Anlam
`00`	RR (Receive Ready)	Mesaj almaya hazırım.
`01`	REJ (Reject)	Şu numaralı çerçeveyi alamadım ya da hatalı aldım, yeniden gönder.
`10`	RNR (Receive Not Ready)	Alıcı şu an hazır değil.
`11`	SREJ (Selective Reject)	Yalnız belirtilen çerçeveyi yeniden gönder.

SREJ için hocanın örneği: 3 numaralı çerçeve alınamamış ama 7'ye kadar alınmış. 3 numaralı çerçevede hata var. Seçici ret yapılacaksa kod alanına 11, N(R) alanına da reddedilen çerçevenin numarası yazılır.

Hoca, RR, REJ, RNR ve SREJ basit olduğu ve bilinmesini istediği için S-frame'i ayrıntılı anlattığını, U-frame'de bu kadar ayrıntıya girmeyeceğini söyledi.

15.2 RR'nin Kullanımları

S-frame bir RR ise onay (ACK) için kullanılabilir.

Birincil cihaz P bitini 1 yaparak RR gönderirse bu bir **poll**'dur. Hocanın örneğinde denetim alanı `1 0 0 0` ile başlar: `10` S-frame, `00` RR. P biti 1'dir ve adres alanında ikincil cihazın numarası, örneğin 3 yazar. Yani 3 numaralı cihaza, P'si 1, kodu `00` olan bir S-frame birincilden gider ve poll anlamına gelir. Python ile kurulduğunda bu denetim alanı, N(R) = 0 için diyagram sırasıyla `10001000` olur.

İkincil cihaz poll'u reddediyorsa, yani gönderecek bir şeyi yoksa, aynı diziyi F biti 1 olarak gönderir. P ve F aynı bittir; hangi tarafın gönderdiğine göre P ya da F denir.

Birincil cihaz select ile "sana veri göndereceğim" diye talepte bulunabilir. İkincil bunu kabul etmek istiyorsa yine RR ile F = 1 gönderir.

Hoca burada önemli bir noktayı vurguladı: "Poll'u reddet" ile "select'i kabul et" arasında bit dizisi olarak hiçbir fark yoktur. İkincil, birincilden gelen mesajın içeriğine cevap verir. Birincil cevabı açıp "bu poll'u mu reddetti, select'i mi kabul etti?" diye şaşırmaz, çünkü kendisinin ne sorduğunu bilir.

Poll örneği: Birincil RR ve P = 1 gönderir. İkincilin verisi varsa veriyi S-frame ile değil I-frame ile gönderir. I-frame'in denetim kısmındaki F biti 0 ise "veriyi parçalara bölerek gönderiyorum, bu parça son parça değil" demektir. F = 1 ise "bu son parça, bu veriyle ilgili daha parça göndermeyeceğim" demektir. Poll kabul edilince veri karşıdan gelmeye başlar. Poll reddedilirse "sana gönderecek hiçbir şeyim yok" denmiş olur.

15.3 RNR'nin Kullanımları

Kod `10` ise çerçeve RNR'dir ve üç kullanımı vardır:

- onay için: "şu ana kadar aldım ama şu an hazır değilim",
- select yapmak için,
- select'i reddetmek için.

Özetle:

- Birincilden ikincile **RNR + P = 1**: "Sana bir şey göndereceğim, uygun musun?" (select).
- Birincilden ikincile **RR + P = 1**: "Bana göndermek istediğin bir şey var mı?" (poll).
- İkincilden **RR + F = 1**: select'e cevapsa "tamam, kabul ettim".
- İkincilden **RNR + F = 1**: select'i reddediyor.

  Ders sonrası not: Hoca poll'u anlatırken bir yerde RR'yi "0-1", RNR'yi "1-0" diye andı. Kodlar RR = `00`, REJ = `01`, RNR = `10`, SREJ = `11`'dir. Hoca ayrıca ikincilin kabul cevabını bir yerde "receive ready ile beraber P1" diye söyledi; ikincilden giden bit F'dir.

15.4 REJ ve SREJ: Go-Back-N ve Seçici Ret

Hoca iki ARQ türünü kısaca tekrar etti.

- **Go-Back-N ARQ:** Bir çerçeve hatalı geldiyse, ondan sonra başka çerçeveler de gelmiş olabilir. Alıcı onların hatalı mı hatasız mı geldiğine hiç bakmadan "3 numaralı çerçeve hatalı geldi, 3'ten sonrasını yeniden gönder" der. Bu REJ'dir.
- **Selective Reject ARQ:** Örneğin 7'ye kadar çerçeveler gelmiş, 4 numaralı çerçeve hatalı gelmiş. Alıcı SREJ 4 yapınca gönderici yalnız 4'ü gönderir. Sonra gönderilenlerde sorun yoktur ya da onlarla ilgili mesaj daha sonra iletilecektir.

16. U-Frame

Denetim alanı 11 ile başlıyorsa U-frame'dir. Oturumun denetimi için çeşitli yönetim işlerinde kullanılır. Üçüncü bitte yine bir P/F biti vardır. Bir bayt 8 bit olduğuna göre 3 bit çıkınca geriye 5 bit kalır: 2 bit bir yerde, 3 bit başka bir yerde. Bu 5 bitle $2^5 = 32$ farklı yönetim işlemi tanımlanabilir.

```
1 1 | kod 2 bit | P/F | kod 3 bit
```

Bu 32 işlem beş kategoride toplanır:

- **Mod seçme:** İletime başlamadan önce iletimin nasıl olacağı U-frame ile karşı tarafa söylenir. 5 bitlik değer bir diziyse "normal response modda çalışacağım, pencere 7" (SNRM), başka bir diziyse "asenkron dengeli modda çalışacağım, pencere 7" (SABM) denir. Bunların pencere 127 olan sürümleri de vardır (SNRME, SABME). Mod, veri gönderilmeye başlanmadan önce U-frame ile söylenir.
- **Bilgi değişimi:** Örneğin poll isteği ya da zaman ayarı. "Hadi senkronize olalım" demek için belirli bir dizi gönderilir. Bu bir komut olabilir ("saatini şuna ayarla"; ardından bilgi alanında saat gönderilebilir) ya da "saatin kaç?" sorusuna gelen bir cevap olabilir. Bir U-frame ile istenen yönetim işleminin sonucunda "saatini şuna ayarla dediğini aldım" gibi cevaplar da gönderilebilir.
- **Bağlantı kesme (disconnection):** Üç tip bağlantı sonlandırma yöntemi vardır. 32 dizinin hepsine bit kombinasyonunu söylemek yerine daha kolay anlaşılsın diye kısa adlar verilmiştir: UA, UI, SNRM gibi. İletişimi ilk başlatan taraf her şey bitince bağlantıyı sonlandıracaksa DISC gönderir. İkincil cihazın "bağlantıyı artık sonlandır" demek için gönderdiği bir dizi de vardır.
- **Başlatma (initialization):** "Sana veri göndermeye başlayacağım" demek için birincilden ikincile gönderilir. Bir de "senden başlatma bekliyorum" anlamına gelen hâli vardır.
- **Diğer işler, örneğin sıfırlama (reset):** Bir bağlantıda çeşitli sorunlar yüzünden pencerede hangi yerde olunduğu, hangi çerçevenin alındığı birbirine karışmış olabilir. "Bu işi baştan başlatalım" demek için özel bir dizi vardır.

16.1 Aynı Kod, Farklı Anlam

Bağlantı kesme satırlarında bir öğrenci tablodaki iki satırın kodlarının aynı olduğunu fark etti. Hoca önce slaytta bir yanlışlık olduğunu, kodların aynı olmaması gerektiğini söyledi. Başka bir öğrenci, birinin birinci taraftan, öbürünün ikinci taraftan gönderildiğini, bu yüzden kodun aynı olabileceğini söyledi. Hoca slayttaki açıklamaya baktı: Biri, iletişimi ilk kuran istasyon tarafından bağlantıyı sonlandırmak için gönderiliyordu. Hoca başlatma satırlarında da aynı durumu görünce öğrencinin doğru söylediğini kabul etti: Aynı dizi birincilden gelince bir anlama, ikincilden gelince başka bir anlama gelir. Başlatma satırlarında biri "başlatmayı başlatıyorum", öbürü "senin başlatmanı bekliyorum" demektir.

> **Ek bilgi (derste anlatılmadı):** Büyük olasılıkla HDLC standardında bu çiftler şöyledir: DISC (Disconnect, komut) ile RD (Request Disconnect, cevap) aynı kodu kullanır; SIM (Set Initialization Mode, komut) ile RIM (Request Initialization Mode, cevap) da aynı kodu kullanır. Komut ya da cevap olduğu, çerçevenin hangi istasyondan geldiğinden anlaşılır.
>
> **Hocanın vurgusu:** U-frame bit dizilerini ezberlemeyin. "Şu bit dizisi ne anlama gelir?" diye ayrıntılı bir soru hiçbir zaman sorulmaz. Kabaca ne tür işler yapıldığını (mod seçme vb.) bilmek önemlidir. Hoca kendisinin de hepsini bilmediğini, gerektiğinde açıp baktığını söyledi.

Kabaca özet: U-frame yönetim işleri için kullanılır. I-frame ile veri gönderilir; I-frame içinde piggyback onay da gönderilebilir. S-frame ile verinin alınıp alınmadığı ya da hangilerinin alındığı bildirilir.

17. Poll/Final Biti

Mesaj birincilden ikincile gidiyorsa bu bite **poll (P)** biti denir. İkincilden birincile geliyorsa **final (F)** biti denir. Her iki durumda da bit 1 olabilir; adı yalnız mesajın nereden nereye gittiğine göre değişir.

18. HDLC İşleyiş Örnekleri

HDLC’de iletişim üç çerçeve tipiyle üç evreli bir mekanizmayla yürür:

- bağlantının kurulması,
- akış denetimi kullanılarak verinin karşı tarafa aktarılması,
- bağlantının sonlandırılması.

18.1 Bağlantı Kurma ve Kesme

A tarafı B’ye bir şeyler gönderecek. Önce bağlantıyı başlatmak için, kullanılacak modu da söyleyen bir mesaj gönderir. Bayrak standarttır: bir 0, altı tane 1, bir 0. Çerçeve B’ye gider. Bunun bir U-frame olduğu denetim alanının ilk iki bitinin 1 olmasından anlaşılır; slaytta anlaşılsın diye ayrıca “U-frame” yazılmıştır. Bu mesajda P = 1’dir.

İkincil B “tamam, bağlantıyı başlatabiliriz” diye cevap verir. Bu kez F = 1 gelir ve kod alanındaki değerler farklıdır. Ardından aktif bir bağlantı vardır. Bu bağlantıda veriler gelip gider, reset gelebilir; U-frame’ler, S-frame’ler, I-frame’ler gidip gelir. Sonunda başlatan taraf olarak A, DISC ile sonlandırma talebinde bulunur. B de sonlandırmayı kabul eden bir cevap gönderir.

18.2 Piggybacking ile Çift Yönlü Veri Aktarımı

Bu örnekte iki taraf da veri gönderir. Her I-frame’in denetim alanında “hangi çerçeveyi gönderiyorum” (N(S)) ve “senden hangisini bekliyorum” (N(R)) yazar.

Gönderen	Çerçeve	N(S)	N(R)	Anlamı
A	I	0	0	0 numaralı çerçeveyi gönderiyorum, senden 0’ı bekliyorum.
B	I	0	1	0’ı gönderiyorum, senin 0’ını aldım, 1’i bekliyorum.
A	I	1	1	1’i gönderiyorum, senin 0’ını aldım, 1’i bekliyorum.
A	I	2	1	2’yi gönderiyorum, senden hâlâ 1’i bekliyorum.
B	I	1	3	1’i gönderiyorum, 1’i de 2’yi de aldım, 3’ü bekliyorum.
A	S (RR)	–	2	Gönderecek verim kalmadı; 1’e kadar aldım, 2’yi almaya hazırım.

Akış sırayla gitmek zorunda değildir. Bir taraf daha fazla gönderebilir. Hocanın açıklaması: B aynı anda C ile de konuşuyor olabilir ve hep A’ya vakit ayıramayabilir, ama A hep B ile konuşuyordur. Bu yüzden A art arda iki çerçeve gönderir. B’nin “3’ü bekliyorum” demesi, 1’i

de 2'yi de aldığı anlamına gelir; 1 için ayrıca onay göndermez. Kayan pencerenin mantığı budur. Son satırda A'nın B'ye gönderecek verisi kalmadığı için onay I-frame ile değil, S-frame ile gider.

Python ile her tarafın aldığı son çerçeve izlenerek tablodaki N(R) değerleri yeniden üretildi; hepsi tuttu.

18.3 Alıcının Hazır Olmaması

Bu örnek bir önceki slaytın devamı değildir; başka bir veri gönderiminin parçası gibi düşünülmelidir.

- A, B'ye I-frame gönderir: "3'ü gönderiyorum, senden 0'ı bekliyorum." A'nın B'den daha hiçbir şey almamış olması mümkündür.
- B, S-frame ile cevap verir: "Sana gönderecek bir şeyim yok, 3'ü aldım, 4'ü de alacağım ama almaya henüz hazır değilim" (RNR, N(R) = 4).
- A, S-frame ile "bana göndermek istediğin bir şey var mı?" diye sorar (RR, P = 1). B yine "hayır, yok; en son 3'ü aldım, hazır değilim" der.
- A bir kez daha sorar. Bu kez B RR ile gelir: "3'e kadar olanları almıştım, önceden hazır değildim, şimdi almaya hazırım."
- B hazır olduğu için A 4 numaralı I-frame'i gönderir.

Alıcı neden hazır olmayabilir? Hocanın açıklaması: Çok noktalı bir yapıda B, A ile konuşurken başka biriyle de konuşuyor olabilir ve onun önceliği daha yüksek olabilir. Arka tarafta başka işlemler yürütüyor olabilir. Tamponu dolmuş olabilir ve temizlemeye çalışıyor olabilir.

18.4 Kaybolan Çerçeve: REJ

Durum her zaman "güllük gülistanlık" değildir. Gönderilen çerçeve yolda bir sorun yüzünden karşıya ulaşmayabilir ve alıcının bundan haberdar olması gerekir.

- A, B'ye I-frame'lerle 3, 4 ve 5 numaralı çerçeveleri gönderir; senden 0'ı beklediğini de söyler.
- 4 yolda kaybolur. Dışarıdan bakan bunu görür, ama B o zaman diliminde göremez.
- B en son 3'ü almıştır, şimdi 5 gelir. "4 nerede?" diye eksikliği anlar.
- B hemen S-frame ile REJ 4 gönderir: "4'ü almadım; 4'ü ve ondan sonra gönderdiklerin varsa hepsini gönder."
- A, 4'ten itibaren hepsini yeniden gönderir.

B, gelen çerçevenin 5 numaralı olduğunu anladığı an REJ gönderir. 5'in içindeki veriyi açıp hata denetimi yapmakla hiç uğraşmaz.

REJ yerine SREJ yapılsaydı B "4 numaralı çerçeve, seçici ret, yalnız 4'ü yeniden gönder" derdi. 5'i bu arada kendisi açıp hata denetimini yapardı. 5'i başarıyla aldıysa bir süre sonra onu da aldığını bildirirdi. Almamış da olabilir; bir çerçevenin bütün bitlerinin doğru geldiği ancak açıp hata denetim teknikleri uygulandıktan sonra anlaşılır.

Python ile kontrol: Gönderilen 3, 4, 5 ve kaybolan 4 için REJ ile yeniden gönderilenler 4 ve 5, SREJ ile yalnız 4'tür.

18.5 Çok Noktalı Hatta Poll

Birincil istasyon ve iki ikincil istasyon (A ve B) vardır.

- Birincil A'ya poll yapar: S-frame ile "bana göndermek istediğin bir şey var mı?" diye sorar (RR, P = 1).
- A'nın gönderecek verisi vardır ve I-frame'lerle gönderir.
- Hoca burada önemli bir noktaya dikkat çekti: Gönderilecek veri örneğin 64 MB'lik bir fotoğraf olabilir. Tek parçada gitmez, birkaç parçaya bölünür. İkincilden birincile giderken bite P değil F denir. I-frame'de F = 0 ise "bu, büyük verinin son parçası değil, daha göndereceğim", F = 1 ise "bu son parça" demektir.
- Birincil A'ya "2'ye kadar olanların hepsini başarıyla aldım" der ve olayı sonlandırır.

  Hocanın vurgusu: Bu kısımda son parçada F'nin 1 olmasına önemle dikkat edin.

Soru: İkinci ve üçüncü satırda adres olarak A yazılmış. Orada birincilin adresi yazılması gerekmiyor mu? **Cevap:** Hayır. Birincil-ikincil ilişkisinde birincil zaten bellidir; hangi cihazın birincil olduğunu bütün ikinciller bilir. Adres alanı, birincile gelen mesajın nereden geldiğini, yani A'dan geldiğini söyler. Birincilin birçok ikincili vardır. A ile B haberleşecekse bu da birincil üzerinden olur: A birincile "B'ye bir şey göndereceğim, sor bakalım müsait mi?" der. Birincil B'ye sorar. B müsaitse birincil A'ya "müsaitmiş, gönder" der. A birincile gönderir, birincil B'ye gönderir.

18.6 Çok Noktalı Hatta Select

- Birincil, RNR ile B'yi "sana veri göndereceğim" diye seçer (select).
- B, S-frame ile "senden mesaj almaya hazırım" der (RR). İkincilin hakkı vardır: Birincil "sana mesaj göndereceğim" dediğinde "hazırım" ya da "hazır değilim" diyebilir.
- İki yönde de adres alanında B yazar. Birincil B'ye gönderirken adres B'dir; ikincil birincile gönderirken de "ben B'yim" der. İlişki çok noktalıdır ama birincil ile ikincil arasında noktadan noktaya gibi düşünülebilir.
- Ardından birincil B'ye I-frame ile veri gönderir. Veri bittiğinde P biti 1 olacaktır.
- B'nin birincile gönderecek verisi olmadığı için S-frame ile onay verir: "Senden 0'ı aldım, 1'i bekliyorum."

Soru: Bu onay I-frame ile de yapılabilir miydi? **Cevap:** Yapılabilirdi, ama I-frame, B'nin de aynı anda birincile cevap olarak bir şey gönderdiği durumda kullanılır. Örneğin konuşuyorsak: o bana sesini gönderiyor, ben ona gönderiyorum. O zaman I-frame'in içinde piggybacking ile "senden şunu aldım" denebilirdi. Burada B'nin birincile gönderecek hiçbir şeyi yoktu. Bu yüzden yalnız gelenleri aldığını bildiren onay ya da alamadığını bildiren ret mesajı S-frame ile gider.

18.7 Piggybacking Tekrarı

Bir öğrenci piggybacking'in ne olduğunu kaçırdığını söyleyince hoca yeniden anlattı. Gönderici karşıya bir mesaj gönderir. Alıcı "mesajı aldım", "hatalı aldım" ya da "şunu gönder" gibi bir bilgilendirme mesajı yollar. Bu sistemde veri göndermek için I-frame, "aldım", "yeniden gönder", ret gibi işler için S-frame kullanılır. Bir önceki bayt yönelimli protokol yarı çift yönlüydü, burada ise tam çift yönlü çalışılır: İki taraf da birbirine veri gönderir. Piggybacking'de gönderilecek I-frame'in içine, sanki bir S-frame gömülüyormuş gibi, "şuraya kadar olanları aldım" bilgisi konur. Hoca bunu domuzun sırtına (kuyruğuna) binen küçük bir yük benzetmesiyle anlattı: Domuz kocaman, kuyruğu ufacıktır. ACK'yı I-frame'in içine yedirmeye piggybacking denir.

18.8 Son Örnek: U-Frame ile Başlayan Tam Akış

Hoca son slaytı kısaca geçti ve öğrencilerin kendilerinin bakmasını istedi. Önce U-frame ile başlatma yapılır, karşı taraf "tamam, başlatabiliriz" der. Ardından veriler gider gelir; adreslere bakınca kimin gönderdiği değişir. A, B'ye 0 numaralı çerçeveyi gönderir. B, A'ya hem "senden aldım" der hem de kendi verisini ekleyip gönderir. En sonda bir S-frame ile "artık senden alacağım hiçbir veri yok" denir.

Örnek Sorular

Bu derste video karesi yoktur. Aşağıdaki sorular derste sözlü sorulan ya da okunan sorulardır.

1. (derste sözlü, 00:26:05) Sohbet kutusundan gelen bir soru: Asenkron iletimde her baytın başına ve sonuna denetim bitleri eklenirken senkron iletimde ek yük nasıl karşılaştırılır?

> **Çözüm (derste verildi):** Asenkron iletimde her bayta başlangıç ve bitiş bitleri eklenir; hoca kaybın %30–40'a çıkabildiğini söyledi. Senkron iletimde çerçevenin veri kısmı 1000–1500 bayt olabilir ve yalnız başlangıç için birkaç bayt gönderilir. Her bayta denetim baytı eklenmez, yalnız çerçevenin nerede başladığını söylemek yeter. Bu yüzden senkron iletim çok daha elverişlidir. (Oranlarla ilgili düzeltme 3.2'deki ders sonrası notta.)

2. (derste sözlü, 00:41:51) Hoca sınıfa sordu: Bayt yönelimli bir protokolde sonu ETB ile biten bir çerçeve alındığında ne anlarız?

> **Çözüm (derste verildi):** Gönderici veri gönderiyor ama daha da gönderecek; çünkü çerçevenin sonu ETX değil. ETX ile biten çerçeve metnin bittiğini gösterir.

3. (derste sözlü, 00:42:12) Ara bloklu çerçevelerde her blok için başlık bilgileri farklı mı?

> **Çözüm (derste verildi):** Başlıktaki bilgiler verinin içeriğiyle ilgilidir. Veriye göre aynı da olabilir, farklı da; net bir şey söylenemez.

4. (derste sözlü, 01:32:44) U-frame tablosunda bağlantı kesme için iki satırın kodu aynı. Bu bir yanlışlık mı?

> **Çözüm (derste verildi):** Hoca önce yanlışlık olduğunu düşündü. Başka bir öğrencinin açıklamasıyla aynı U-frame dizisinin birincilden gelince bir anlama, ikincilden gelince başka bir anlama geldiğini kabul etti. Başlatma satırlarında da aynı durum vardır.

5. (derste sözlü, 01:43:33) Piggybacking nedir?

> **Çözüm (derste verildi):** Tam çift yönlü iletişimde, alıcının "şuraya kadar aldım" onayını ayrı bir S-frame ile değil, kendi gönderdiği I-frame'in N(R) alanına koyarak yollamasıdır. ACK'yı I-frame'in içine yedirmektir.

6. (derste sözlü, 01:52:37) Çok noktalı poll örneğinde, A'nın birincile gönderdiği çerçevelerde adres alanına neden A yazılmış? Birincilin adresi yazılmamalı mı?

> **Çözüm (derste verildi):** Birincil bellidir ve bütün ikinciller onu bilir. HDLC'de adres alanı her zaman ikincilin adresini taşır; birincil bu sayede mesajın hangi ikincilden geldiğini anlar.

7. (derste sözlü, 01:56:00) Select örneğinde B'nin onayı S-frame yerine I-frame ile verilebilir miydi?

> **Çözüm (derste verildi):** Verilebilirdi, ama I-frame ancak B'nin de birincile gönderecek verisi varsa kullanılır ve onay piggybacking ile onun içine konur. B'nin gönderecek verisi olmadığı için onay S-frame ile gider.

Dersin Toparlaması

Bu derste veri bağlantı denetim protokolleri iki ana grupta incelendi. Asenkron protokoller her bayta başlangıç ve bitiş bitleri eklediği ve hatta boşluklar bıraktığı için ek yükü yüksek ve yavaştır. Senkron protokoller ise çerçevenin başını ve sonunu bayraklarla belirtir ve uzun çerçevelerde ek yükü çok düşüktür. Bayt yönelimli senkron protokollerde ASCII denetim karakterleriyle (SYN, SOH, STX, ITB, ETB, ETX, EOT, ACK0/ACK1 vb.) veri ve denetim çerçeveleri kurulur. İkili veri gönderilince ortaya çıkan veri saydamlığı sorunu DLE kaçış karakteriyle çözülür. Bu ailenin en bilinen örneği IBM'in yarı çift yönlü, dur ve bekle kullanan, kod bağımlı BSC protokolüdür.

Bit yönelimli tarafta ISO standardı HDLC ayrıntılı işlendi: istasyon tipleri, dengeli ve dengesiz yapılandırmalar, NRM, ARM ve ABM modları; bayrak, adres, denetim, bilgi ve FCS alanları; bayrağın veride görünmesini engelleyen bit doldurma; I-frame'deki N(S)/N(R) ile piggybacking; S-frame'deki RR, REJ, RNR, SREJ kodları ve P/F biti ile poll/select; yönetim işleri için U-frame. Son olarak bağlantı kurma, veri aktarma, alıcının hazır olmaması, kaybolan çerçeve ve çok noktalı poll/select örnekleri izlendi. Gelecek hafta LAN teknolojilerine, yani IEEE 802 projesine geçilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.